Project Report

On

“Clone and Fake App Detection”

Submitted by

Name: Lokesh Poudyal

Submitted to

BharatCares ACA

IBM SkillsBuild

Table of Contents

S.NO.	CONTENTS	PAGE NO.
	Acknowledgement	3
1.	Introduction	4
2.	Problem Statement	4
3.	Objectives	4
4.	Methodology	5
4.1.	Technologies Used	5
4.2.	System Design Process	5
4.2.1.	Sequence Diagram	5
4.2.2.	Data Flow Diagram (DFD)	6
4.2.3.	Flow Chart	7
4.2.4.	State Transition Diagram (STD)	8
5.	Output	9
5.1.	GUI and APK Analysis *(with code snippet)*	9
5.2.	CSV Log *(with code snippet)*	13
5.3.	Error Log	14
6.	Conclusion	15

Acknowledgment

For all the efforts behind the paper work and project, I first and foremost like to express my heartfelt gratitude to **BharatCares ACA, IBM SkillsBuild**, for his extended help and suggestions for this cybersecurity project work. It is with a great sense of gratitude that I acknowledge the unwavering support, time to time recommendations and guidance throughout the session.

With due regards

Lokesh Poudyal

1. Introduction

Nowadays Android devices are widely used, therefore the risk of encountering fake or cloned apps have grown significantly higher. These malicious apps often mimic legitimate ones, posing threats such as data theft, malware installation, or unauthorized access to user data. The Fake and Clone App Detection application is a cybersecurity tool designed to combat this issue by analyzing android package (APK) files for signs of fabrication or malicious intention. Application is built using python language which features a graphical user interface (GUI) designed with customtkinter, APK analysis capabilities via androguard library, and malware scanning through the VirusTotal API.

The tool is designed to operate locally on user's computer, but requires an internet connection in order to use VirusTotal API for malware checks. User can scan for APK's hash, metadata and potential red flags without internet connection just by using functionalities of androguard library which is embedded into the tool. Tool aims to be effective and user-friendly catering to broader audience that includes casual Android users, developer, and security enthusiasts, who need a straightforward way to verify app authenticity before installation. By analyzing parameters such as APK's hash, metadata, permissions, VirusTotal checkups, and potential red flags, application provides actionable insights into whether an app might be counterfeit or malicious.

2. Problem Statement

Detecting and mitigating clone and fake apps.

The app stores are flooded with fake versions of popular apps. These clone apps are designed to trick users into thinking they're downloading the real thing. Once installed, they can cause serious harm like stealing your passwords, spreading viruses, or locking your phone. Even though there are security measures in place, fake apps are still a big problem. This shows that we need a simpler and more effective way to help users spot these dangerous apps.

Source: https://static.mygov.in/innovateindia/2025/01/14/mygov-1000000000685961903.pdf

3. Objectives

The Fake and Clone App Detection tool was designed with following goals:

- To create an intuitive interface. Design a simple, visually appealing GUI that allows users to upload APK files and view results with ease of use.
- To perform comprehensive APK analysis. Allow tool to generate a unique hash, extract metadata (e.g. permissions, package details), and detect suspicious indicators of cloning or fakery.
- To leverage external malware checks. To integrate the VirusTotal API to validate APKs against a database of known threats, enhancing reference or auditing.
- To log results clearly and concisely. Allow tool to display scan results in GUI and log them in CSV file with applicable parameters for future reference or auditing.
- To ensure robustness and privacy. Build a reliable tool that handles errors effectively and contains an error log, protects user data by sending only hash value to external API and maintains operational stability.

4. Methodology

While developing the Fake and Clone App Detection application we followed a systematic approach, which includes different phases such as coding, system design, and testing. This section explains the process involved during designing and coding phase with diagrams of sequence diagram, data flow diagram (DFD), flow chart, and state transition diagram (STD).

4.1. Technologies Used:

Programming Language: Python programming
Key Libraries/Tools:

- Androguard: For APK metadata extraction.
- VirusTotal API (requests): For file reputation checking (malware database).
- CustomTkinter: For an interactive, visually appealing GUI.
- CSV Module: For logging scan results.
- hashlib: Generates a unique SHA-256 hash for the APK file.
- cryptography: Processes and analyzes digital certificates embedded in the APK.

4.2. System Design process

In order to showcase the application's structure and behavior, four diagrams were designed on the project's workflow. The application's core functionality is built around following the steps involved each diagram which are explained below with its respective subsection.

4.2.1. Sequence Diagram

The sequence diagram (shown in figure 1) outlines the interactions between different components during APK file analysis process. The components used in sequence diagram includes User, GUI, APKAnalyzer, VirusTotal API, and CSV Logger, which are self-explanatory. This diagram explains the chronological order of actions and dependencies between the components used, and threading functionality is being used in application to maintain GUI's responsiveness during analysis. The process involves:

- User clicks "Browse APK" and selects APK file.
- User clicks "Start Scan", which prompts GUI to call APKAnalyzer.
- APKAnalyzer generates hash, extracts metadata, queries VirusTotal API, and checks for fake indicators.
- APKAnalyzer compiles the results and sends them to GUI for displaying to user.
- GUI triggers CSV Logger to save all scan results, showing the path in confirmation dialog for user.

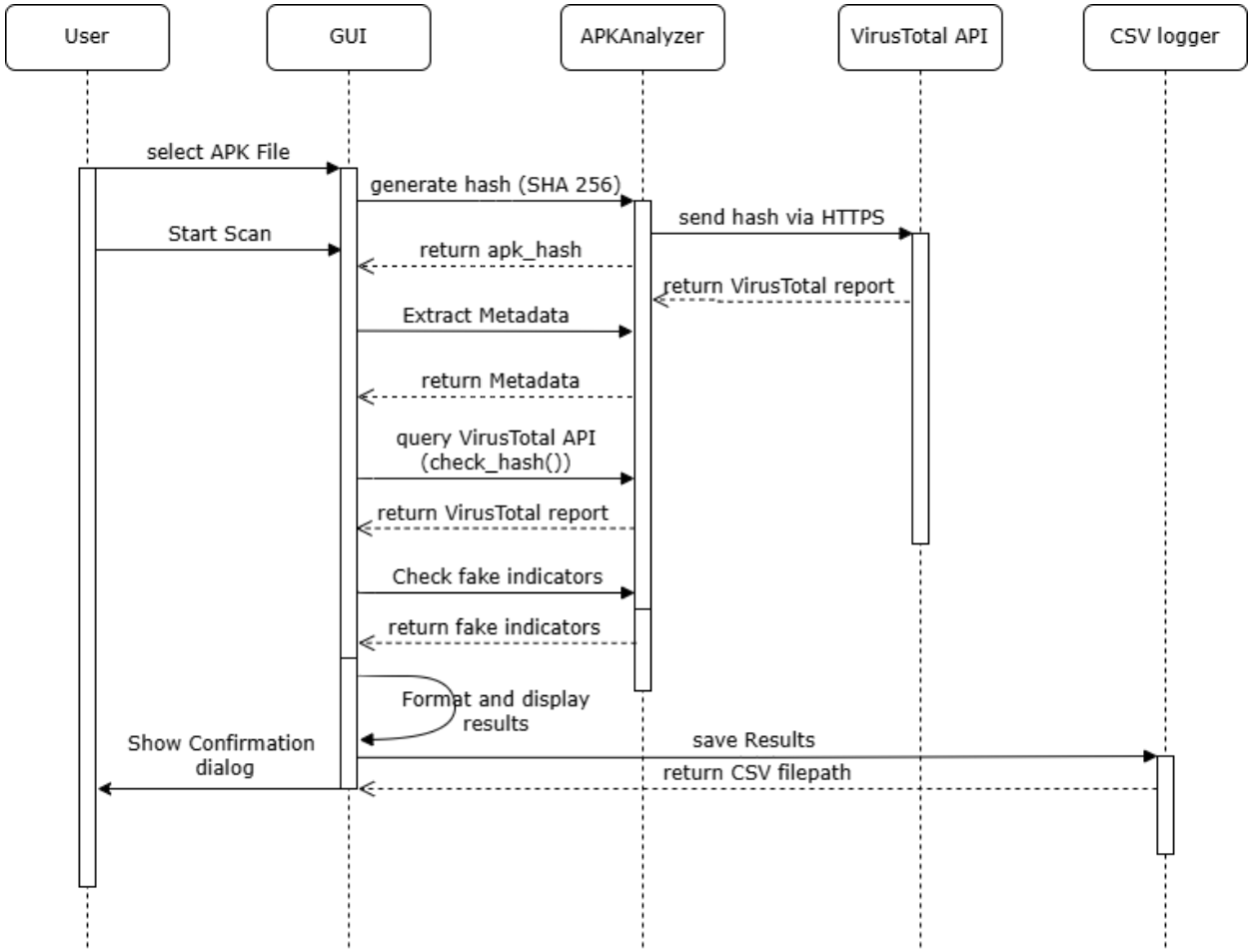


Figure 1: Sequence diagram for Fake and Clone App Detection Application

4.2.2. Data Flow Diagram (DFD)

Data flow diagram (DFD) (shown in figure 2) shows the flow of data through the system, from input to output. DFD contains components such as External Entities, processes and Data stores to showcase the data flow in the system. External entities include User and VirusTotal API which supplies the APK and receives analysis report, and supplies malware scan data respectively. APK File and CSV log are created as Data stores that holds selected APK, and stores analysis records respectively. DFD emphasizes data transformation, showing how APK evolves into detailed report through sequential processing and external validation.

Explanation of Data Flows is shown as:

- APK path flows from User to Select APK File process, the into APK File store.
- APK is processed to produce a hash, which is used into metadata extraction and the VirusTotal query.
- Results from data processes are combined and saved in CSV log, and displayed to User.

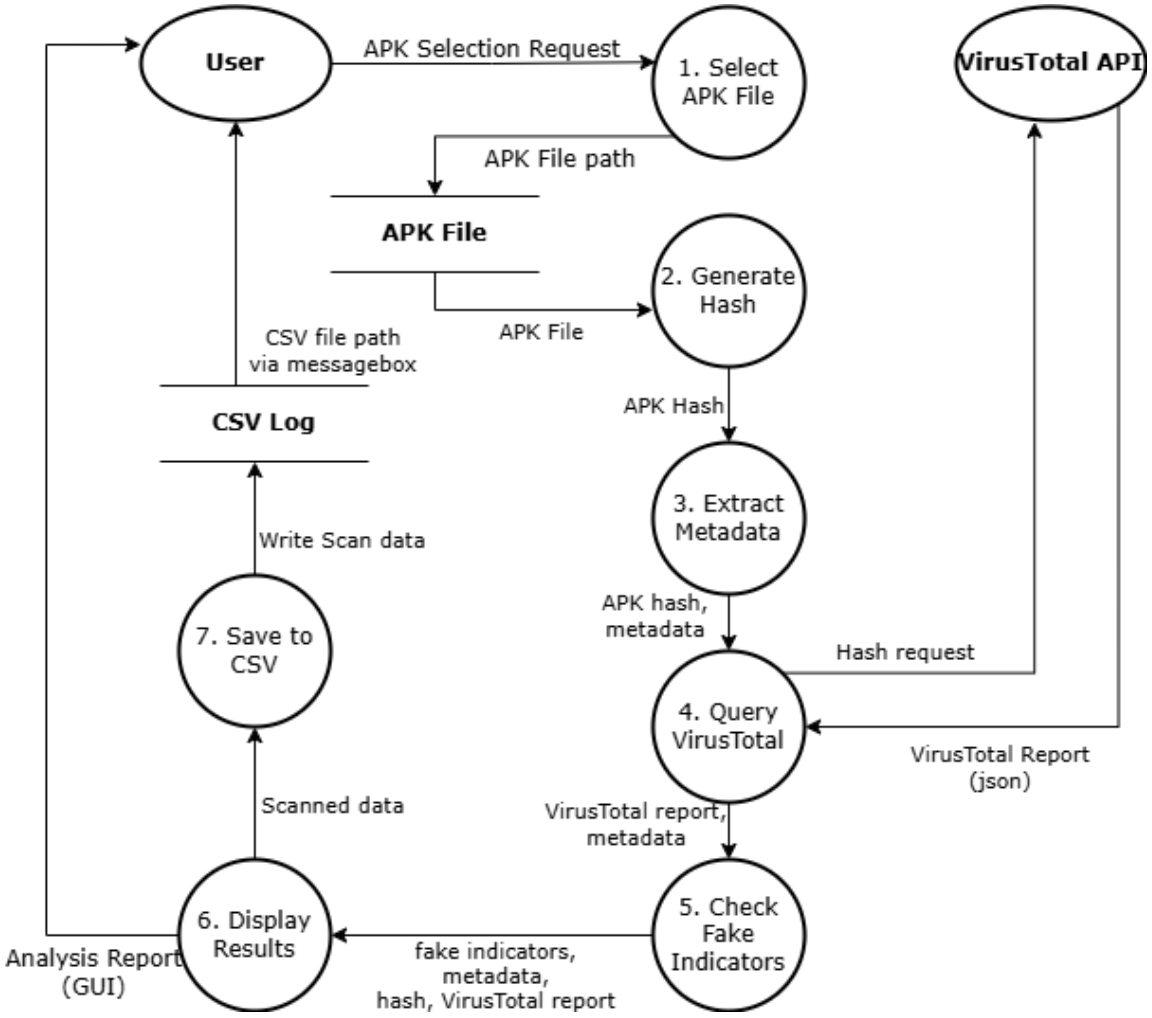


Figure 2: Data flow diagram (DFD) of Fake and Clone App Detection

4.2.3. Flow Chart

The Flow Chart (shown in figure 3) provides a step-by-step visual guidance on how application works for analyzing APK file. Steps involved in flow chart is:

- APK File Selection: User browses and selects an APK file.
- Hash Generation: Generate a SHA-256 hash.
- Metadata Extraction: Use Androguard to retrieve package info, permissions, certificates, etc.
- VirusTotal Query: Check generated APK's hash against VirusTotal's database.
- Fake Indicator Analysis: Detect suspicious permissions and missing certifications.
- Display results and error logs: Results appear in the GUI and are logged to a CSV file. Log for errors during execution of application is also generated.

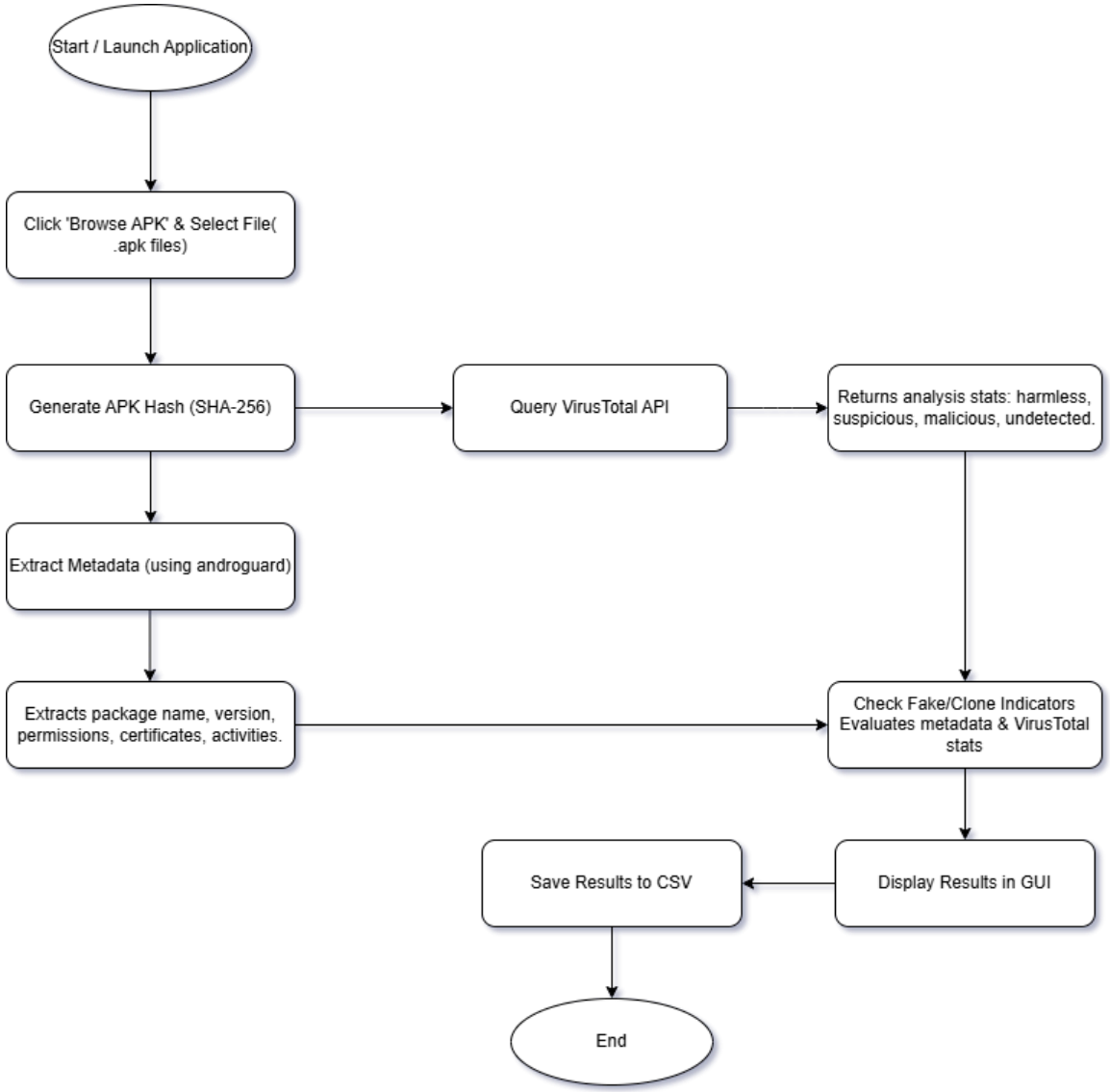


Figure 3: Flow chart for Fake and Clone App Detection

4.2.4. State Transition Diagram

State Transition Diagram (STD) (shown in figure 4) explains the application's states and transitions triggered by different events throughout the working of application. States in STD has been defined with different parameters such as Idle, selecting file, Analyzing, Displaying Results, Saving Log and Error. STD captures application's dynamic behavior showing the way it transitions between different states based on user inputs and system events. Transitions in STD shown as:

- Idle to selecting file when user clicks "Browse APK".
- Selecting file to Analyzing when user clicks "Start Scan".
- Analyzing to Displaying Results when analysis completes successfully.
- Displaying Results to Saving when scanned result is saved to CSV.
- Saving Log to Idle when save operation terminates.
- Any state to Idle during any occurrence of errors (e.g., no file selected)

- Error to Idle after user acknowledges the error.

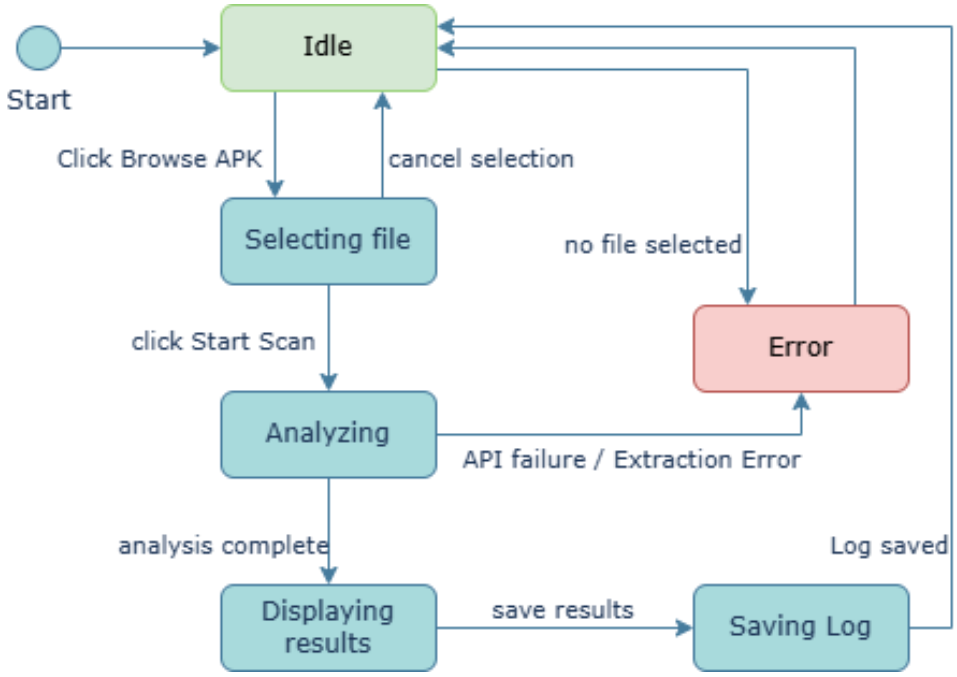


Figure 4: State Transition Diagram for Fake and Clone App Detection

5. Output

The developed Fake and clone app detection application successfully scans and analyzes APK files and fulfills its objectives. The user interface (as shown in figure 5.1 and 5.2) allows users to easily upload and initiate scans along with basic instructions on how to use the application. For instance, when scanning Vidmate.apk, the system identified suspicious app permissions (as shown in the figure 6), which indicates potential risks. Furthermore, scanning GB Whatsapp.apk flagged the application as malicious through VirusTotal API analysis (as shown in figure 7), confirming its malicious status.

Furthermore, the application performs comprehensive APK metadata extraction using Androguard, generates a unique SHA-256 hash, and verifies digital certificates to authenticate app developers. All scan results, including error logs and key parameters, are concisely stored in a CSV file for future reference and further analysis.

5.1. GUI and APK analysis

User-friendly Interface is designed for simplified APK upload and scanning. GUI features sections for instructions, input field, a progress bar, and text bar results. GUI (as shown in figure 5.1) is designed in the initial phase using only single frame and GUI shown in figure 5.2 is designed as final design for the application in which left panel is created for instructions and right panel for input fields and text box results, highlighting its user-friendly design and key elements.

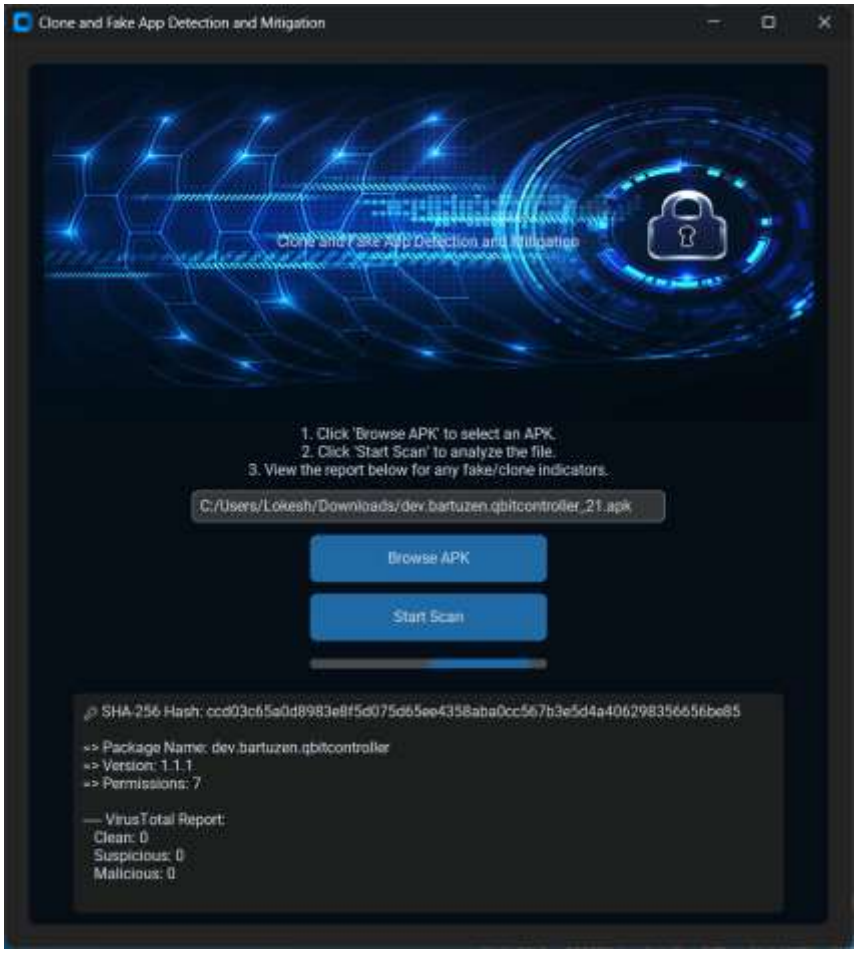


Figure 5.1: Initial design for Fake and Clone App Detection

Below is the code snippet for final design (as shown in figure 5.2) for the interface:

```
ctk.set_appearance_mode("dark")
ctk.set_default_color_theme("blue")

self.app.title("Clone and Fake App Detection")
self.app.geometry("900x700")

self.main_frame = ctk.CTkFrame(self.app, fg_color="#121212")
self.main_frame.pack(pady=10, padx=10, fill="both", expand=True)

self.left_frame = ctk.CTkFrame(self.main_frame, fg_color="black", corner_radius=20)
self.left_frame.pack(side="left", fill="both", expand=True, padx=10, pady=10)

self.right_frame = ctk.CTkFrame(self.main_frame, fg_color="#0e074a", corner_radius=20)
self.right_frame.pack(side="right", fill="both", expand=True, padx=10, pady=10)
```

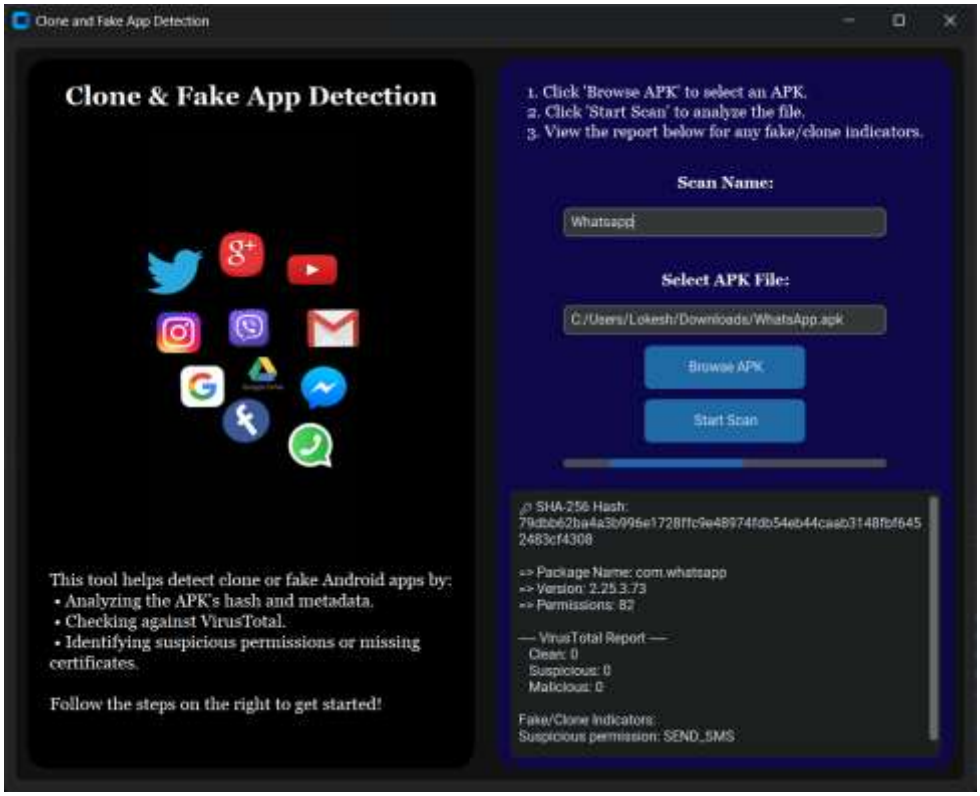


Figure 5.2: Final design for Fake and Clone App Detection

Metadata and Permission Analysis: Detailed inspection using Androguard (shown in figure 6). The *extract_metadata* function retrieves APK details using Androguard. Code snippet is given below:

```
def extract_metadata (apk_path: str) -> dict:
    try:
        a, _, _ = AnalyzeAPK(apk_path)
        certs = []
        for cert in a.get_certificates():
            try:
                der_data = cert.get_raw()
                x509_cert = load_der_x509_certificate(der_data)
                public_key = x509_cert.public_key().public_bytes(
                    encoding=Encoding.PEM,
                    format=PublicFormat.SubjectPublicKeyInfo
                ).decode()
                algorithm = x509_cert.signature_algorithm_oid.dotted_string
            except Exception:
                public_key = "Unknown"
                algorithm = "Unknown"

            certs.append({
                'public_key': public_key,
                'signature_algorithm': algorithm
```

```
        })
    return {
        'package_name': a.get_package(),
        'version': a.get_androidversion_name(),
        'permissions': a.get_permissions(),
        'certificates': certs,
        'activities': a.get_activities()
    }
except Exception as e:
    logger.error(f"Metadata extraction failed: {e}")
    return {}
```

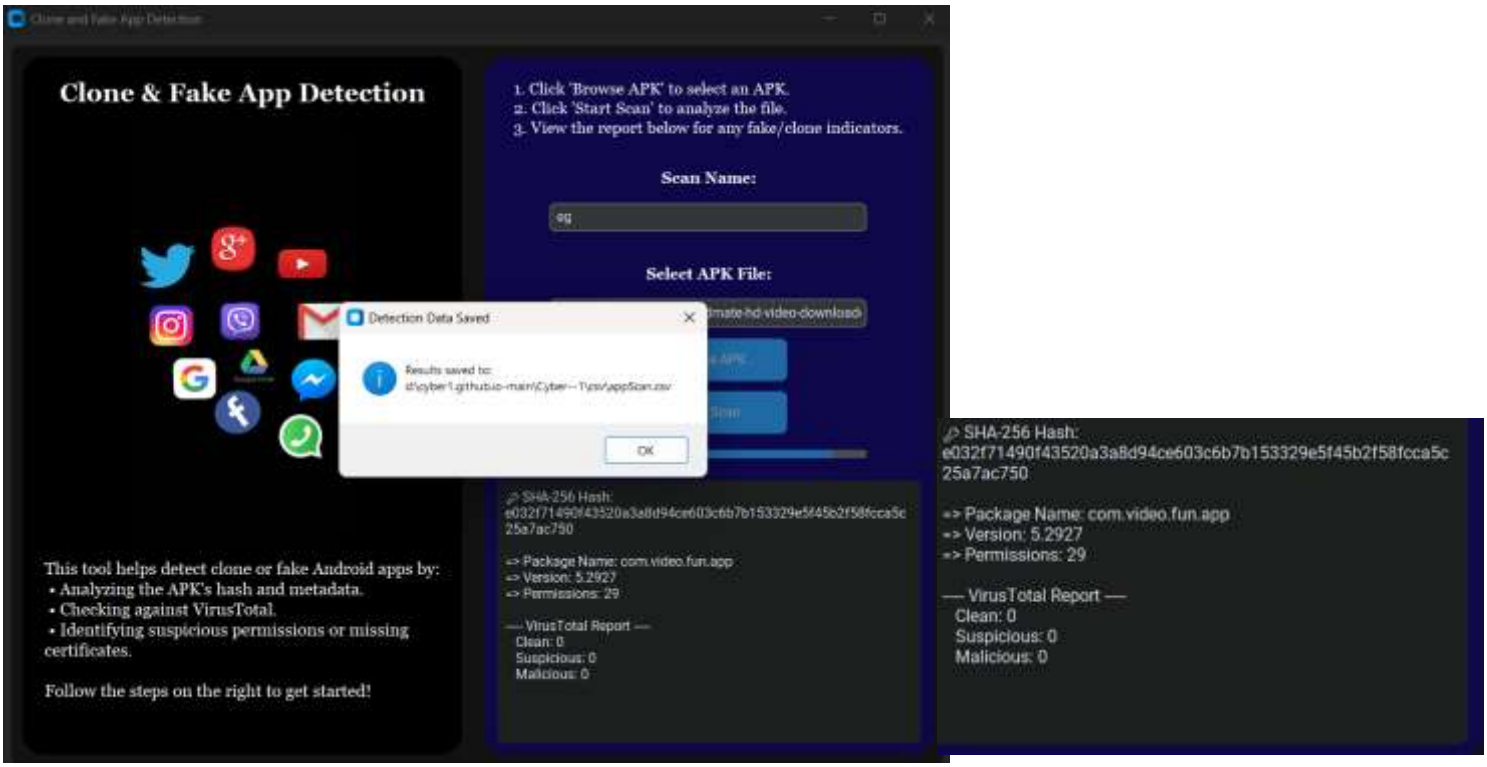


Figure 6: scan details of vidmate.apk (Excess Permissions)

Security Checks: VirusTotal API effectively detects malicious apps (as shown in figure 7). The result confirms malicious status based on VirusTotal's database. The *VirusTotalClient* class queries VirusTotal, code snippet for this is as follows:

```
class VirusTotalClient:
    def __init__(self):
        self.base_url = "https://www.virustotal.com/api/v3"
        self.headers = {"x-apikey": VIRUSTOTAL_API_KEY}

    def check_hash(self, file_hash: str) -> dict:
```

```
# check file hash against VirusTotal database
try:
    response = requests.get(
        f"{self.base_url}/files/{file_hash}",
        headers=self.headers
    )
    response.raise_for_status()
    return response.json()
except requests.exceptions.RequestException as e:
    logger.error(f"API request failed: {e}")
    return{}
```

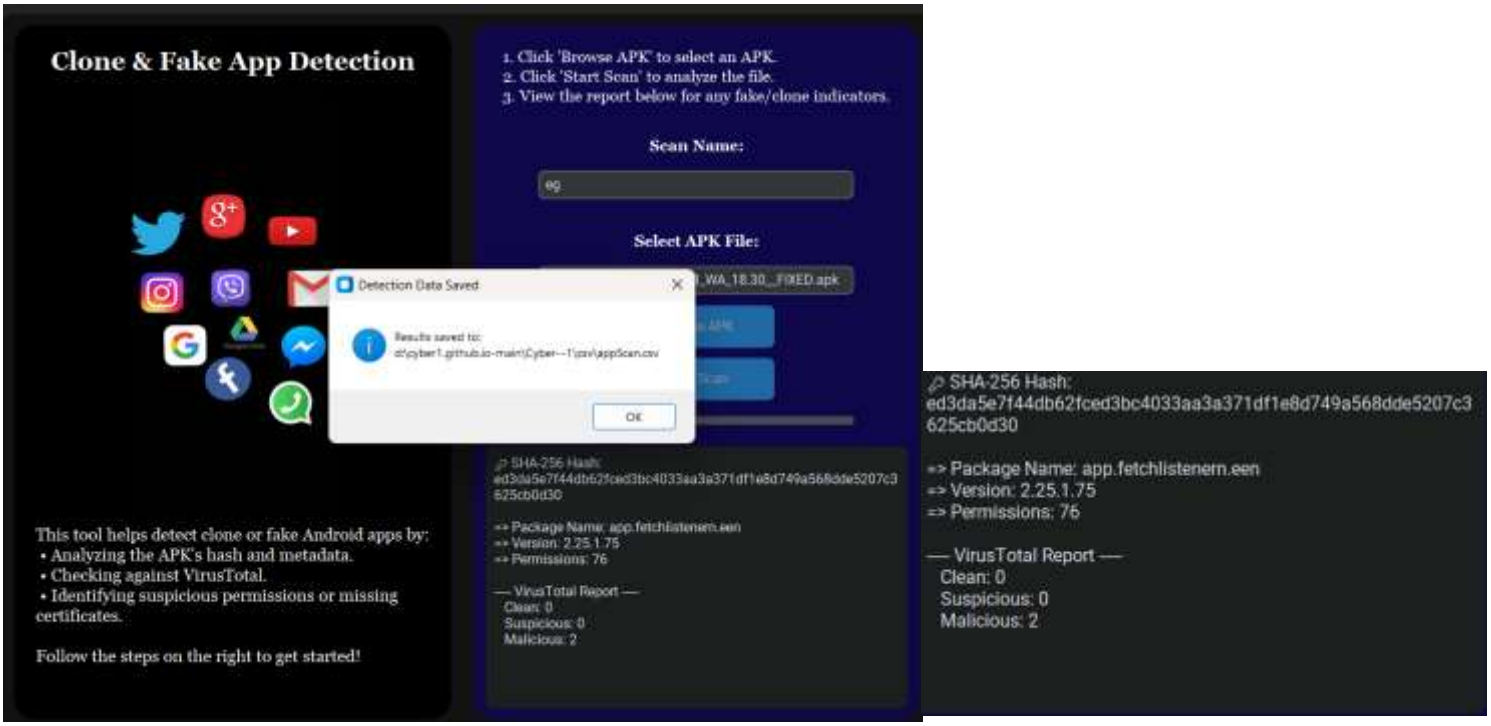


Figure 7: scan details of gbwhatsapp.apk(malicious)

5.2. CSV Log

Secure storage of scan details and error logs for future reference. The appScan.csv file logs each scan with columns with different parameters. Scan results are logged with *detection_data* function:

```
def detection_data( scan_name: str, apk_path: str, apk_hash: str, metadata: dict, vt_report: dict, fake_indicators: list,  output_csv: str = "csv/appScan.csv") -> str:
    file_exists = os.path.isfile(output_csv)

    package_name = metadata.get('package_name', 'N/A')
    version = metadata.get('version', 'N/A')
    permissions_count = len(metadata.get('permissions', []))
```

```
    vt_data = vt_report.get('data')
    if vt_data:
        attributes = vt_data.get('attributes', {})
        stats = attributes.get('last_analysis_stats', {})
        vt_clean = stats.get('harmless', 0)
        vt_suspicious = stats.get('suspicious', 0)
        vt_malicious = stats.get('malicious', 0)
    else:
        vt_clean = 0
        vt_suspicious = 0
        vt_malicious = 0

    indicators_str = ", ".join(fake_indicators) if fake_indicators else "None"
    timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
     row = [timestamp, scan_name, apk_path, apk_hash, package_name, version, permissions_count,
vt_clean, vt_suspicious, vt_malicious, indicators_str ]
    with open(output_csv, mode='a', newline='', encoding='utf-8') as csv_file:
        writer = csv.writer(csv_file)
        if not file_exists:
            writer.writerow([
                "Timestamp", "Scan Name", "APK Path", "SHA-256 Hash", "Package Name", "Version",
"#Permissions", "VT Clean", "VT Suspicious", "VT Malicious", "Fake/Clone Indicators" ])
        writer.writerow(row)
    return os.path.abspath(output_csv)
```

	A	B	C	D	E	F	G	H	I	J	K
1	Timestamp	Scan Nam	APK Path	SHA-256 Hash	Package Name	Version	#Permissi	VT Clean	VT Suspici	VT Malicio	Fake/Clone Indicators
2	15-02-2025 10:58	Kul	C:/Users/L	ccd03c65a0d8963e8f5d075d65ee4358eba0cc567b3e5d4a	dev.bartuzen.qbitcontroller	1.1.1	7	0	0	0	None
3	15-02-2025 11:08	Whatsapp	C:/Users/L	79dbb62ba8a3b696e1728ffc9e48974fdb54eb44caab3148fb	com.whatsapp	2.25.3.73	82	0	0	0	Suspicious permission: SEND_SMS
4	15-02-2025 11:16	Clone Wha	C:/Users/L	5797205f2e19110a95fb508f3ad1cf618b89054aa1ef655b67	clone.whatsapp	2.0.0	2	0	0	0	None
5	15-02-2025 11:22	Yalpstore	C:/Users/L	75977af2866af4b5aa51e360f6e7b8f3972ccd90a816ea02d	com.github.yeriomin.yalpstore	0.27	6	0	0	0	None
6	19-02-2025 12:16	ZArch	C:/Users/h	19c92cbe4d7c5edf67d1e175dc64adc88c8fa132e56d5185e	ru.zdevs.zarchiver	1.0.10	10	0	0	0	None
7	19-02-2025 12:40	Un-named	D:/note 10	4245af1af3e7dd80c56a273597ef3c75e3e5ced67386e1bc7	com.picsart.studio	23.2	28	0	0	2	None
8	07-03-2025 11:28	Un-named	D:/note 10	e15d4fd0931211d63a51ca625bb8e7fbcb8bc69d6bff1fc3d8	org.adaway	6.1.1	9	0	0	0	None
9	07-03-2025 11:35	Un-named	D:/note 10	dc3bb88f6419ee7dde7d1547a41569aa03282fe00e0dc43ce	org.fdroid.fdroid	1.17.0	23	0	0	0	None
10	18-03-2025 22:05	eg	D:/note 10	e032f71490f43520a3a8d9dce603c6b7b153329e5fd5b2f58f	com.video.fun.app	5.2927	29	0	0	0	None
11	18-03-2025 22:07	eg	D:/note 10	ed3da5e7f44db62fced3bc4033aa3a371df1e8d749a568dde	app.fetchlistenern.een	2.25.1.75	76	0	0	2	None
12											

Figure 8: scan details logged using different parameters in csv file

5.3. Error Log

Error handling ensures robustness. Errors like missing files are logged and displayed.

```
Cyber-1 > $ app_detection.log
1   2025-02-12 19:39:55,536 - ERROR - Metadata extraction failed: 'Certificate' object has no attribute 'der_cert'
2   2025-02-12 19:39:56,177 - ERROR - Analysis failed: 'VirusTotalClient' object has no attribute 'check_hash'
3   2025-02-12 20:08:33,916 - ERROR - Metadata extraction failed: 'Certificate' object has no attribute 'der_cert'
4   2025-02-12 20:21:54,599 - ERROR - Metadata extraction failed: 'Certificate' object has no attribute 'get_der'
5   2025-02-12 20:38:30,849 - ERROR - Metadata extraction failed: 'Certificate' object has no attribute 'get_public_key'2025-02-15 06:36:37,132 - WARNING -
6   2025-02-15 06:37:18,229 - WARNING - Could not load image: module 'PIL.Image' has no attribute 'ANTIALIAS'
7   2025-02-15 06:38:30,107 - WARNING - Could not load image: module 'PIL.Image' has no attribute 'ANTIALIAS'
8   2025-02-15 06:41:15,417 - WARNING - Could not load image: module 'PIL.Image' has no attribute 'ANTIALIAS'
9   2025-02-15 06:42:35,049 - WARNING - Could not load image: module 'PIL.Image' has no attribute 'ANTIALIAS'
10  2025-02-15 06:58:58,242 - WARNING - Could not load image: [Errno 2] No such file or directory: 'wallpaper.jpeg'
11  2025-02-15 10:58:38,383 - ERROR - API request failed: HTTPSConnectionPool(host='www.virustotal.com', port=443): Max retries exceeded with url: /api/v3/f
12  2025-02-15 11:15:02,103 - ERROR - Analysis failed: [Errno 13] Permission denied: 'csv/appScan.csv'
```

Figure 9: Error logs saved

6. Conclusion

The Fake and Clone App Detection application provides practical solution for detecting fake or malicious Android applications. The application is Python-based designed using customTkinter GUI, androguard for local APK analysis, VirusTotal API for external malware validation, while detailed system diagrams (STD, DFD, Flow Chart and sequence diagram) provide a clear blueprint of its design and operation.

The application met its objectives after its strong performance during testing. Intuitive GUI for navigation, identification of suspicious apps, and reliable operation made the application to deliver its functional foundation which were validated through different outcomes with detailed snapshots. Potential improvements for application could include adding more fake and clone detection criteria and conditions in code, adding additional external APIs such as Hybrid Analysis, or integrating machine learning for enhanced accuracy, behavioral analysis, and deep metadata scrutiny. This project addresses a critical cybersecurity issue by tackling proliferation of counterfeit Android applications and also establishes a framework for further improvements due its open design collaboration.